Java SE 17 Programmer II（1Z0-826-JPN）　試験内容 4 Java モジュール・システム　#11

モジュールの移行とサービス

名前付き・自動・無名モジュール、ボトムアップとトップダウン、jdeps、ServiceLoader

各ページの下に、根拠にした資料（Oracle の試験内容チェックリスト、JEP 261、Java SE 17 API の ModuleFinder・ServiceLoader、JLS 7.7.3・7.7.4、JDK 17 Tool Specifications の jdeps・jar）を書いた。jar・モジュールの結果は JDK 17（Eclipse Temurin 17.0.21）で実行して確かめた

全体像 どこに置くかで、モジュールの種類が決まる

	名前付きモジュール	自動モジュール	無名モジュール
置く場所	モジュール・パス	モジュール・パス	クラスパス
module-info	ある	ない	ない（あっても無視）
名前	module-info の名前	Automatic-Module-Name かファイル名	なし（getName() は null）
公開	exports したものだけ	すべて	すべて
読む	requires したものだけ	すべて（無名も）	すべて
requires できるか	○	○	×（名前がない）
jlink	○	×	×

名前付きモジュールはクラスパスを読めない：package legacy.util does not exist。

出典：JEP 261（Automatic modules・Unnamed modules）、API Module.getName・isNamed／JDK 17 でのコンパイルと実行の結果

第1章

自動モジュールと無名モジュール

名前の決まり方と、読める範囲。

自動モジュールの名前

jar のファイル名	名前@バージョン
commons-lang3-3.12.0.jar	commons.lang3@3.12.0
my-util.jar	my.util
my_util-2.0-SNAPSHOT.jar	my.util@2.0-SNAPSHOT
legacy.tools-1.0.jar	legacy.tools@1.0
x--y-1.jar	x.y@1
named-1.0.jar+Automatic-Module-Name: com.example.legacy	com.example.legacy@1.0

ファイル名から作る手順

① .jar を取る

② 末尾の「-数字…」をバージョンとして外す

③ 英数字以外をドットに

④ ドットの連続を 1 つに、前後のドットを取る

> マニフェストの Automatic-Module-Name があれば、そちらが優先。

出典：API ModuleFinder.of（automatic module の名前）／jar --describe-module の結果（No module descriptor found. Derived automatic module.）

読める範囲

名前付き com.three
requires bridge;

↓ requires

自動モジュール bridge
bridge.jar（モジュール・パス）

↓ 自動モジュールは無名モジュールも読む

無名モジュール
-cp depout の other.O

```
java -p out3:mp3 -cp depout -m com.three/com.three.Main
// other
java -p out3:mp3 -m com.three/com.three.Main
// NoClassDefFoundError: other/O
```

分割パッケージ

```
mp/my-util.jar              legacy.util
mp/commons-lang3-3.12.0.jar legacy.util
// error: module com.app reads package legacy.util
//   from both my.util and commons.lang3
```

クラスパスのクラス

```
Hello.class.getModule()          // unnamed module @58dc58f9
Hello.class.getModule().isNamed() // false
Hello.class.getModule().getName() // null
```

出典：JEP 261、API Module／JDK 17 でのコンパイルと実行の結果

第2章

移行と jdeps

下から進めるか、上から進めるか。

ボトムアップとトップダウン

ボトムアップ（依存される側から）

app.jar
まだ → クラスパス（無名）

↓ 使う

service
module-info を足した → モジュール・パス

↓ 使う

util
最初にモジュール化

無名はすべてを読めるので、上の層はクラスパスのままでよい。

トップダウン（アプリから）

com.app
module-info を足した

↓ requires my.util;

my-util.jar
まだ → モジュール・パスに置いて自動モジュール

自分で直せないライブラリがあるときに向く。

出典：JEP 261、The State of the Module System（Migration）

jdeps

```
jdeps -s cpout plainout
cpout -> java.base
cpout -> plainout
plainout -> java.base
jdeps --module-path mp2 -s out/com.app
com.app -> java.base
com.app -> my.util
jdeps --generate-module-info gen my-util.jar
module my.util {
    exports legacy.util;
}
```

```
jdeps --jdk-internals jiout
jiout -> jdk.unsupported
  Int -> sun.misc.Unsafe  JDK internal API (jdk.unsupported)
JDK Internal API     Suggested Replacement
sun.misc.Unsafe      See http://openjdk.java.net/jeps/260
```

オプション	中身
-s（-summary）	まとまり単位の依存
--jdk-internals	JDK 内部 API の利用と代替
--generate-module-info	module-info.java のひな形
--generate-open-module	open module のひな形

出典：JDK 17 Tool Specifications jdeps、JEP 260／JDK 17 での実行結果

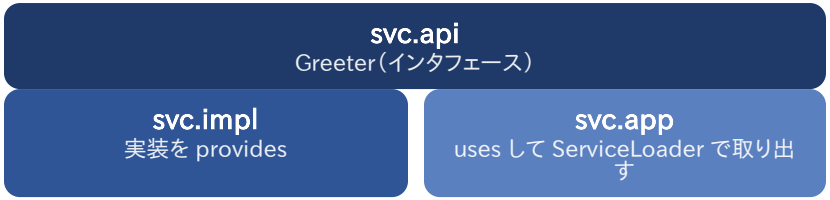
第3章

サービス

uses・provides と ServiceLoader。

3 つのモジュールの役割

```
module svc.api {                  // SPI
  exports svc.api;
}
module svc.impl {                 // プロバイダ
  requires svc.api;
  provides svc.api.Greeter
      with svc.impl.Polite, svc.impl.Casual;
}
module svc.app {                  // 使う側
  requires svc.api;
  uses svc.api.Greeter;
}
```



> svc.app は svc.impl を requires しない。svc.impl は実装のパッケージを exports しない（contains svc.impl）。

出典：JLS 7.7.3 Service Consumption・7.7.4 Service Provision／java --describe-module の結果

ServiceLoader

```
ServiceLoader<Greeter> sl = ServiceLoader.load(Greeter.class);
for (Greeter g : sl) println(g.greet("A") + " " + simpleName);
// Hello, A Polite
// [provider]Hi A Casual   ← provider() が呼ばれた
sl.stream().map(p -> p.type().getSimpleName()).toList()
// [Polite, Casual]   ← type() はインスタンスを作らない
ServiceLoader.load(Greeter.class).findFirst()  // Optional
```

状況	結果
実装がない	空（stream は []、findFirst は empty）
uses がない	ServiceConfigurationError（does not declare `uses`）
実装が private コンストラクタだけ	コンパイルエラー（not public）
インタフェースを実装していない	コンパイルエラー（must be a subtype …）
public static provider() がある	それが呼ばれる（コンストラクタは private でよい）

出典：API ServiceLoader・ServiceLoader.Provider・ServiceConfigurationError、JLS 7.7.4／JDK 17 でのコンパイルと実行の結果

本番での問われ方

ねらい	見るところ
モジュールの種類	置く場所（-p か -cp）と module-info の有無
自動モジュールの名前	Automatic-Module-Name 優先、ハイフン → ドット、末尾のバージョンを外す
読めるか	名前付きは無名を読めない、自動はすべてを読む、分割パッケージはエラー
移行	ボトムアップは下から＋残りはクラスパス、トップダウンは上から＋ライブラリは自動モジュール
jdeps	-s・--jdk-internals・--generate-module-info
サービス	uses は使う側、provides … with は実装側、API は exports、使う側は実装を requires しない

出典：試験内容チェックリスト 4／本資料の問題演習（全 30 問、JDK 17 で確認）